COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	Sep 29, 2026	**DATE SUBMITTED:**	Sep 29, 2026
NAME:	1. Mapas, Dustin Carl 2. Nacion, Daniela Joy 3. Panti, Rodnee 4. Paraiso, Wayne Kaisen 5. Payas, John Paul	**INSTRUCTOR:**	Engr. Jimlord Quejado

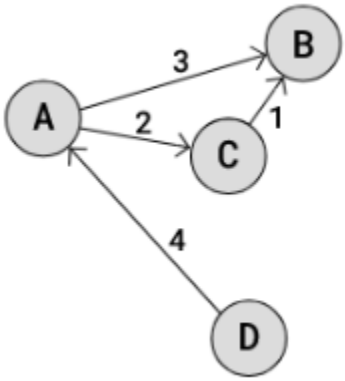
ACTIVITY NO. 10.1 **Seatwork 10.1 - Graph**

1. PROCEDURE OUTPUT

1. Define the following:

Graph - It is a non-linear data structure consisting of vertices connected by edges.
Vertex - It is an individual fundamental point in a graph.
Nodes - It is the data containing points in a graph
Weight - It is a numerical value assigned to an edge representing cost, distance, or capacity.
Path - It is a sequence of connected edges that joins a sequence of vertices.
Directed graph - It is a graph where all edges have a specific direction.
Connected graph - It is a graph in which a path exists between every pair of vertices.
Directed cyclic - It is a graph containing at least one path that starts and ends at the same vertex.
Adjacency matrix - It is a 2D array representation where matrix cells indicate edge presence and weights between pairs of vertices.
Adjacency List - It is a collection of linked lists or arrays where each vertex stores a list of its directly connected neighboring vertices.

2. Identify the parts of the following graph.



Vertices: A, B, C, and D

Directed Edges:
A → B with weight 3
A → C with weight 2
C → B with weight 1
D → A with weight 4

3. Plot the adjacency matrix of the graph above.

	A	B	C	D
A	0	3	2	0
B	0	0	0	0
C	0	1	0	0
D	4	0	0	0

4. Create a program in C++ program to create a graph that looks like in the above figure.

```cpp
#ifndef GRAPH_H
#define GRAPH_H

#include <iostream>
#include <vector>

template <typename T>
class Graph {
private:
    std::vector<char> vertices;
    std::vector<std::vector<T>> adjMatrix;

public:
    Graph(const std::vector<std::vector<T>>& matrix) : adjMatrix(matrix) {
        size_t size = adjMatrix.size();
        vertices.resize(size);

        for (size_t i = 0; i < size; ++i) {
            vertices[i] = static_cast<char>('A' + i);
        }
    }

    void display() const {
        std::cout << "Adjacency Matrix representation of the graph:\n\n";

        std::cout << "    ";
        for (size_t i = 0; i < vertices.size(); ++i) {
            std::cout << vertices[i] << "  ";
        }
        std::cout << "\n";

        for (size_t i = 0; i < adjMatrix.size(); ++i) {
            std::cout << vertices[i] << "   ";
            for (size_t j = 0; j < adjMatrix[i].size(); ++j) {
                std::cout << adjMatrix[i][j] << "  ";
            }
            std::cout << "\n";
        }
    }
};

#endif
```

Graph.h

```cpp
#include "Graph.h"
#include <vector>

int main() {
    std::vector<std::vector<int>> adjacencyMatrix = {
        {0, 3, 2, 0},
        {0, 0, 0, 0},
        {0, 1, 0, 0},
        {4, 0, 0, 0}
    };

    Graph<int> myGraph(adjacencyMatrix);

    myGraph.display();

    return 0;
}
```

matrix.cpp

```
Adjacency Matrix representation of the graph:

   A  B  C  D
A  0  3  2  0
B  0  0  0  0
C  0  1  0  0
D  4  0  0  0
```

Result

5. Differentiate BFS vs DFS types of search.

BFS	DFS
Traverses the graph level by level.	Traverses the graph depth-first.
Uses a Queue (FIFO).	Uses a Stack (LIFO) or recursion.
Visits neighboring vertices first before proceeding to the next level.	Follows one path as far as possible before backtracking.
Can be used to find the shortest path in an unweighted graph.	Commonly used for path exploration, cycle detection, and connected-component traversal.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.